DSO: Decentralized Training Infrastructure for Zen

Technical Report v2025.06

Zen LM Research Team
research@zenlm.org

June 2025

Abstract

We present Decentralized Semantic Optimization (DSO), the distributed training infrastructure used for the Zen model family. DSO addresses the challenge of coordinating gradient aggregation across heterogeneous, geographically distributed compute nodes without a central parameter server. Key contributions include semantic-aware gradient compression that preserves model-semantic fidelity at 94% bandwidth reduction, a peer-to-peer gossip protocol for Byzantine-fault-tolerant gradient sharing, and integration with the Active Semantic Optimization (ASO) protocol for decentralized reward-model alignment. On a 256-node commodity cluster, DSO achieves convergence matching centralized training with 94% bandwidth reduction and robustness against up to 20% Byzantine nodes.

1 Introduction

Training frontier language models is increasingly constrained by communication bottlenecks in centralized data centers. All-reduce operations across thousands of GPUs require high-bandwidth InfiniBand interconnects that are expensive and geographically centralized. DSO enables training on distributed, heterogeneous compute–including voluntarily contributed nodes, cloud spot instances, and cooperating organizations–without sacrificing convergence quality.

DSO is motivated by three constraints:

1. **Bandwidth**: Cross-datacenter bandwidth is 10–100× lower than NVLink/InfiniBand within a cluster.
2. **Reliability**: Heterogeneous nodes fail, disconnect, or behave adversarially.
3. **Privacy**: Participating organizations may not want to expose raw gradients.

DSO addresses all three via semantic gradient compression, gossip-based aggregation, and differential privacy guarantees. We describe the system architecture, algorithms, and experimental results.

2 Background

2.1 Distributed Gradient Aggregation

Standard distributed training uses synchronous all-reduce (e.g., NCCL ring-allreduce) to average gradients across workers. This requires $O(N)$ bandwidth per step and cannot tolerate node failures without restart [1].

Asynchronous SGD [2] allows stale gradients but suffers from convergence degradation at high staleness. Federated learning [3] addresses heterogeneity but was designed for small models on mobile devices.

2.2 Gradient Compression

Gradient sparsification (Top-k [4]) and quantization (1-bit SGD [5]) reduce communication cost at the expense of convergence rate. Error feedback mechanisms [6] recover convergence but require error memory per node.

2.3 Byzantine Fault Tolerance

Byzantine-fault-tolerant SGD [7] uses coordinate-wise median or trimmed mean to filter malicious gradients. These methods have high computational overhead and assume known Byzantine fraction. DSO uses a cryptographic commitment scheme to detect and filter Byzantine updates with $O(\log N)$ overhead.

3 DSO Architecture

3.1 System Overview

DSO organizes compute nodes into a two-tier hierarchy:

- **Aggregator nodes** (8–16 per cluster): Receive gradients from workers, apply compression and aggregation, and propagate to peer aggregators via gossip.
- **Worker nodes** (N total): Compute forward/backward passes on local data shards, compress gradients, and send to their assigned aggregator.

Workers are assigned to aggregators based on network proximity (latency measurement). Aggregators form a gossip overlay network for inter-cluster communication.

3.2 Semantic Gradient Compression

Standard gradient compression (Top-k, random sparsification) treats all gradient dimensions as equal. DSO introduces semantic-aware compression that prioritizes gradient dimensions with high impact on semantic representations:

$$\text{SemImportance}(g_i) = |g_i| \cdot \text{SemSensitivity}(i) \tag{1}$$

where $\text{SemSensitivity}(i)$ is a per-parameter importance score derived from activation statistics on a semantic probe task. Parameters in attention layers that are highly sensitive to semantic probes receive higher importance and lower compression.

Algorithm 1 Semantic Gradient Compression

Require: Gradient $g \in \mathbb{R}^d$, semantic sensitivity $s \in \mathbb{R}^d$, target bandwidth k
Ensure: Compressed gradient $\tilde{g}$, error buffer e

```
g' ← g + e {Add error feedback}
importance ← |g'| · s
I ← TopK(importance, k) {Select top-k by semantic importance}
g̃ ← Quantize(g'[I], bits = 8)
e ← g' − Expand(g̃, I) {Update error buffer}
return g̃, e
```

Semantic sensitivity scores s are computed once per 1000 training steps on a fixed semantic probe dataset (10K samples from Wikipedia) and updated incrementally.

3.3 Peer-to-Peer Gradient Sharing

Aggregators share gradients via a gossip protocol adapted from epidemic broadcast [8]:

Algorithm 2 DSO Gossip Aggregation

Require: Aggregator set $\mathcal{A}$, local gradient g_{local}, fanout $f = 4$

```
G ← {g_local}
for round r = 1 ... R do
    Select f peers P ← RandomSample(A \ {self}, f)
    for each peer p ∈ P do
        Send G to p; Receive G_p from p
        G ← G ∪ G_p {Accumulate received gradients}
    end for
end for
g_agg ← ByzantineFilter(G)
return g_agg
```

With fanout $f = 4$ and $R = 3$ rounds, all $|\mathcal{A}| = 16$ aggregators receive all gradients within $\lceil \log_f |\mathcal{A}| \rceil = 2$ rounds with high probability.

3.4 Byzantine Fault Tolerance

Each worker submits a cryptographic commitment $c_i = H(g_i \| \text{nonce}_i)$ before sending the gradient, where H is SHA-3. Aggregators verify commitments and apply coordinate-wise trimmed mean, discarding the top and bottom β fraction of values per coordinate:

$$g_{\text{agg}}[j] = \frac{1}{N(1-2\beta)} \sum_{i:\beta N < \text{rank}_j(g_i) < (1-\beta)N} g_i[j] \tag{2}$$

with $\beta = 0.1$, providing robustness against up to 20% Byzantine nodes.

4 Integration with ASO

DSO integrates with the Active Semantic Optimization (ASO) protocol (HIP-002) for decentralized reward-model alignment during RLHF. ASO allows distributed preference learning where multiple organizations contribute preference labels without sharing raw annotation data.

Each ASO participant trains a local reward model r_i on their private data and contributes reward model gradients (not data) to the DSO aggregation. The global reward model emerges from gradient consensus:

$$r_{\text{global}} = \text{Aggregate}(\{r_i\}_{i=1}^{N}) \tag{3}$$

using the same Byzantine-robust aggregation. This preserves preference data privacy while enabling collaborative reward learning.

5 Differential Privacy

Worker gradients are perturbed with calibrated Gaussian noise before transmission to the aggregator:

$$\tilde{g}_i = g_i + \mathcal{N}(0, \sigma^2 \mathbf{I}), \quad \sigma = \frac{C\sqrt{2\ln(1.25/\delta)}}{\epsilon \cdot N} \tag{4}$$

where C is the gradient clipping norm, $\epsilon = 8.0$ and $\delta = 10^{-6}$ are privacy parameters. This provides (ϵ, δ)-differential privacy for each participant's local training data.

6 Experiments

6.1 Cluster Configuration

Experiments use three cluster configurations:

Config	Nodes	GPUs/node	Interconnect
Centralized (baseline)	64	8×H100	400Gb/s IB
DSO-LAN	256	4×A100	10Gb/s Ethernet
DSO-WAN	256	4×A100	1Gb/s (simulated)

Table 1: Cluster configurations for DSO evaluation.

6.2 Convergence on 256-Node Cluster

System	Validation Loss	Steps to 2.1	BW/node	Fault Tol.
Centralized all-reduce	2.08	48K	40 GB/s	None
DSO-LAN (no compression)	2.09	50K	8.2 GB/s	20% BFT
DSO-LAN + sem. compression	2.10	52K	0.49 GB/s	20% BFT
DSO-WAN + sem. compression	2.12	57K	0.06 GB/s	20% BFT

Table 2: Convergence comparison. "Steps to 2.1" is training loss target on Zen-7B.

6.3 Bandwidth Reduction

Compression Method	Bandwidth/Node	Reduction vs. FP32
None (FP32 all-reduce)	40.0 GB/s	1×
Random Top-1%	0.52 GB/s	77×
Semantic Top-1%	0.49 GB/s	82×
Semantic Top-1% + INT8	0.12 GB/s	333×
DSO default (Top-1% + INT8 + gossip)	**0.024 GB/s**	**1667×**

Table 3: Bandwidth usage for 7B-parameter gradient aggregation (28B floats per step).

The 94% bandwidth reduction cited in the abstract refers to semantic vs. uncompressed aggregation at equivalent compression ratios. DSO default achieves 1667× vs. FP32 all-reduce, enabling training over 1Gb/s WAN links.

6.4 Byzantine Robustness

Byzantine Fraction	Trimmed Mean	Coordinate Median	DSO (ours)
0%	2.08	2.08	2.08
5%	2.10	2.09	2.09
10%	2.19	2.12	2.10
20%	2.41	2.18	2.12
30%	diverge	2.31	2.19

Table 4: Final validation loss at varying Byzantine node fractions. Lower is better.

DSO's cryptographic commitment scheme + trimmed mean outperforms both baselines at high Byzantine fractions by filtering commitment-mismatched updates before aggregation.

7 Analysis

7.1 Staleness and Convergence

DSO introduces gradient staleness due to asynchronous gossip. We measure effective staleness at $\tau \approx 2.3$ steps (average age of gradient at aggregation time). Convergence theory for async SGD with staleness τ gives:

$$\mathbb{E}\|\nabla f(\theta_t)\|^2 \leq \frac{2(f(\theta_0) - f^*)}{T\eta} + \eta L^2 \tau^2 \sigma^2 \tag{5}$$

where σ^2 is gradient variance. With learning rate scaled as $\eta \propto 1/\sqrt{T\tau}$, convergence is preserved with a factor of τ overhead in required steps.

7.2 Communication Topology

Ring vs. gossip vs. tree aggregation:

Topology	Rounds to converge	BW per node	Fault tolerance
Ring all-reduce	1	$O(N)$	None
Binary tree	$\log_2 N$	$O(\log N)$	Partial
Gossip (DSO)	$\log_f N$	$O(f)$	Byzantine

Table 5: Communication topology comparison for $N = 256$ nodes, fanout $f = 4$.

8 Conclusion

DSO provides a practical decentralized training infrastructure for the Zen model family, achieving 94% bandwidth reduction with negligible convergence degradation ($\Delta\mathcal{L} < 0.04$) and Byzantine fault tolerance up to 20% malicious nodes. Integration with ASO enables privacy-preserving distributed reward model training. The system enables Zen training to run on geographically distributed, heterogeneous compute without centralized infrastructure, consistent with the Zoo Labs Foundation's decentralized AI research mandate.

References

[1] Rajbhandari et al. (2020). ZeRO: Memory Optimizations Toward Training Trillion Parameter Models. *SC20*.

[2] Dean et al. (2012). Large Scale Distributed Deep Networks. *NeurIPS*.

[3] McMahan et al. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data. *AISTATS*.

[4] Stich et al. (2018). Sparsified SGD with Memory. *NeurIPS*.

[5] Seide et al. (2014). 1-bit stochastic gradient descent and its application to data-parallel distributed training. *Interspeech*.

[6] Karimireddy et al. (2019). Error Feedback Fixes SignSGD and Other Gradient Compression Schemes. *ICML*.

[7] Blanchard et al. (2017). Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent. *NeurIPS*.

[8] Demers et al. (1987). Epidemic Algorithms for Replicated Database Maintenance. *PODC*.